# Lab 07: Inheritance, Types of Inheritance, Final and Super Keyword in Inheritance

**Learning Objectives**

By the end of this lab, you will understand:

- Understand the concept of inheritance in Java.
- Learn single, multilevel, and hierarchical inheritance.
- Understand the use of final methods and final classes.
- Understand abstraction using abstract classes and methods.
- Relate inheritance and abstraction to real-life applications.

**Inheritance:** Inheritance allows one class to acquire the properties and methods of another class.

In Java:

- The existing class is called the parent class or superclass.
- The new class is called the child class or subclass.
- The extends keyword is used for inheritance.

Real-Life Example:

- A Car is a Vehicle.
- A Dog is an Animal.
- A Teacher is a Person.
- A Delivery Robot is a Robot.

Because a child class inherits from a parent class, we can reuse code instead of writing the same variables and methods repeatedly.

**Real-Life Application**

- Student and Teacher classes can inherit from a Person class.
- Car, Bike, and Bus can inherit from a Vehicle class.
- Different types of robots can inherit from a Robot class.
- Online shopping systems can have different product categories inherited from one Product class.

**Single Inheritance:** Single inheritance means one child class inherits from one parent class.

**Code Example**

```
class Vehicle {
   String brand = "Toyota";

   void start() {
      System.out.println("Vehicle is starting");
   }
}

class Car extends Vehicle {
   void display() {
      System.out.println("Brand: " + brand);
   }
}

public class Main {
   public static void main(String[] args) {
      Car c1 = new Car();

      c1.start();
      c1.display();
   }
}
```

**Notes**

- Inheritance promotes code reuse.
- Child classes can use parent class methods and variables.
- Java uses the extends keyword for inheritance.
- A child class can also have its own additional methods.

**Task:** Create a class named Animal with:

- Variable: name
- Method: eat()

Then create a class named Dog that inherits Animal and has an additional method bark().

**Multilevel Inheritance:** Multilevel inheritance means a class inherits from another class, and then another class inherits from it.

**Code Example**

```
class Animal {
   void eat() {
      System.out.println("Animal eats food");
   }
}

class Dog extends Animal {
   void bark() {
      System.out.println("Dog barks");
   }
}

class Puppy extends Dog {
   void weep() {
      System.out.println("Puppy weeps");
   }
}

public class Main {
   public static void main(String[] args) {
      Puppy p1 = new Puppy();

      p1.eat();
      p1.bark();
      p1.weep();
   }
}
```

**Notes**

- A chain of inheritance is created.
- The last child class can access all inherited methods.
- Multilevel inheritance helps represent parent-child-grandchild relationships.

**Task:** Create a multilevel inheritance example using:

- Machine

- Computer
- Laptop

Add one method in each class and call all methods from the Laptop object.

**Hierarchical Inheritance:** Hierarchical inheritance means multiple child classes inherit from the same parent class.

**Code Example**

```
class Shape {
    void draw() {
        System.out.println("Drawing shape");
    }
}

class Circle extends Shape {
    void circleInfo() {
        System.out.println("This is a circle");
    }
}

class Rectangle extends Shape {
    void rectangleInfo() {
        System.out.println("This is a rectangle");
    }
}

public class Main {
    public static void main(String[] args) {
        Circle c1 = new Circle();
        Rectangle r1 = new Rectangle();

        c1.draw();
        c1.circleInfo();

        r1.draw();
        r1.rectangleInfo();

    }
}
```

**Notes**

- One parent class is shared by multiple child classes.
- Each child class can have its own unique methods.
- Hierarchical inheritance is useful when different objects share common behavior.

**Task:** Create a class named Employee with a method work(). Then create two child classes:

- Developer
- Manager

Add separate methods in both child classes.

**Method Overriding:** Method overriding happens when a child class provides its own version of a method already defined in the parent class.

**Real-Life Example:**

- Different robots may move differently.
- Different animals may make different sounds.
- Different vehicles may have different starting behavior.

**Code Example**

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat says Meow");
    }
}
public class Main {
    public static void main(String[] args) {
        Cat c1 = new Cat();
        c1.sound();
    }
}
```

**Notes**

- The method name, parameters, and return type should remain the same.
- Overriding supports runtime polymorphism.
- The @Override annotation is optional but recommended.

**Task:** Create a class named Robot with a method move(). Then create a class named FlyingRobot that overrides move().

**Super Keyword:** Sometimes, a child class and parent class may both have variables or methods with the same name. In such situations, we use the super keyword. The super keyword refers to the immediate parent class object.

**It is commonly used for:**

- Accessing parent class variables
- Calling parent class methods
- Calling parent class constructors

Without super, Java may become confused between the child class members and parent class members.

**Real-Life Example**

- A robot class may have a default speed.
- A delivery robot child class may have its own speed.
- We can use super.speed to access the speed of the parent Robot class.

**Code Example**

```
class Vehicle {
   String brand = "Toyota";
}

class Car extends Vehicle {
   String brand = "Honda";

   void display() {
      System.out.println("Child Brand: " + brand);
      System.out.println("Parent Brand: " + super.brand);
```

```
    }
}

public class Main {
    public static void main(String[] args) {
        Car c1 = new Car();
        c1.display();
    }
}
```

**Using super to Call Parent Class Method**

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }

    void display() {
        sound();
        super.sound();
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d1 = new Dog();
        d1.display();
    }
}
```

**Using super to Call Parent Constructor**

```
class Animal {
    Animal(String type) {
        System.out.println("Animal type: " + type);
    }
}
class Dog extends Animal {
    Dog() {
        super("Mammal");
        System.out.println("Dog constructor called");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

**Notes**

- super refers to the parent class object.
- super.variable is used to access parent variables.
- super.method() is used to call parent methods.
- super() is used to call parent constructors.
- The call to super() should be the first statement inside a child constructor.

**Task:** Create a class named Employee with a variable company. Create a child class named Developer with another variable company. Use super keyword to display both company names.

**Final Method in Inheritance:** A final method cannot be overridden by child classes. This is useful when we want to keep the original behavior fixed.

```
class Bank {
    final void securityRule() {
        System.out.println("Security rule cannot be changed");
    }
}
```

```
class Branch extends Bank {
    void display() {
        System.out.println("Branch information");
    }
}

public class Main {
    public static void main(String[] args) {
        Branch b1 = new Branch();

        b1.securityRule();
        b1.display();
    }
}
```

**Notes**

- Final methods are inherited but cannot be overridden.
- Final methods are useful for security-related logic.
- They help keep important behavior unchanged.

**Task:** Create a class named University with a final method named rules(). Create a child class named Department and call the rules() method.

**Final Class in Inheritance:** A final class cannot be inherited. This is useful when we want to stop further extension of a class.

**Code Example**

```
// Final class cannot be extended
final class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}
// Child class attempting to extend Animal will cause an error
/*
class Dog extends Animal {
    void bark() {
        System.out.println("Woof!");
```

```
    }
}
*/
public class Main {
    public static void main(String[] args) {
        Animal a = new Animal();
        a.eat();
    }
}
```

**Notes**

- Final classes cannot be inherited.
- They are useful for security and preventing unwanted changes.
- Many built-in Java classes are final.

**Exercises:**

**Task 1**

Create a class named Vehicle with:

- Variable: brand
- Method: showBrand()

Create a child class named Bike with:

- Method: showSpeed()

Create an object of Bike and call both methods.

**Task 2**

Create a multilevel inheritance example using:

- Person
- Employee
- Manager

Add one method in each class and call all methods using the Manager object.

**Task 3**

Create a hierarchical inheritance example using:

- Animal
- Cat
- Cow

Add separate methods in Cat and Cow classes.

**Task 4**

Create a class named Shape with a method area(). Create child classes Rectangle and Circle that override the area() method.

**Task 5**

Create a class named School with a final method named discipline(). Create a child class Teacher and call the discipline() method.

**Task 6**

Create a class named School with a final method named discipline(). Create a child class Teacher and call the discipline() method.

**Task 7**

Create a class named Person with a variable name. Create a child class Student with a variable rollNo. Then create another child class GraduateStudent with a variable researchArea. Display all values using the GraduateStudent object.

**Task 8**

Create a class named Vehicle with a method start(). Create two child classes:

- Car
- Bike

Override the start() method in both child classes.

## Mini Project

**Project Title:** Robot Control System

**Problem Statement:** Create a small Robot Control System using the concepts covered in this lab.

**The system should:**

- Use inheritance with a parent Robot class
- Create child classes such as CleaningRobot and DeliveryRobot
- Use multilevel or hierarchical inheritance
- Override methods for different robot behaviors
- Use the super keyword to access parent class values
- Use a final method for safety rules
- Display robot information

**Expected Output**

```
Cleaning Robot Started
Cleaning Robot is moving
Safety Rules Applied

Delivery Robot Started
Delivery Robot is moving
Safety Rules Applied
```